



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9
з дисципліни «Технології розроблення
програмного забезпечення»
Тема: «»
Варіант «Mind-mapping software»

Виконав:

студент групи ІА-32

Бурдін Д. Є.

Перевірив:

Мягкий М. Ю.

Тема:

Мета:

Тема проєкту: Mind-mapping software

Патерни: Strategy, Prototype, Abstract Factory, Bridge, Composite, SOA

Опис: Візуальний додаток для складання "карт пам'яті" з можливістю роботи з декількома картами (у вкладках), автоматичного промальовування ліній, додавання вкладених файлів, картинок, відеофайлів (попередній перегляд); можливість додавання значків категорій / терміновості, обведення областей карти (поділ пунктирною лінією)

Зміст

Теоретичні відомості.....	2
Хід роботи.....	3
Діаграма класів патерну SOA	4
Код програми	4
Висновок	11
Питання до лабораторної роботи	11

Теоретичні відомості

Сервіс-орієнтована архітектура (SOA, англ. service-oriented architecture) – модульний підхід до розробки програмного забезпечення, заснований на використанні розподілених, слабо пов'язаних (англ. Loose coupling) сервісів або служб, оснащених стандартизованими інтерфейсами для взаємодії за стандартизованими протоколами.

Історично сервіс-орієнтована архітектура появилась як альтернатива монолітній архітектурі, в якій вся система розроблялася та розгорталася як одне ціле.

Програмні комплекси, розроблені відповідно до сервіс-орієнтованою архітектурою, зазвичай реалізуються як набір веб-служб (або веб-сервісів), які, як правило, взаємодіють по HTTP з використанням SOAP або REST. Ці служби надають певні бізнес-функції, наприклад, отримання інформації про наявність матеріалів на складі.

Сервіси взаємодіють між собою тільки за рахунок обміну повідомленнями, без створення спеціальних інтеграцій для доступу до однієї інформації, наприклад, до однієї бази даних.

Сервіси також можуть бути реалізовані як обгортки навколо застарілої системи. Це робиться для зменшення вартості переробки системи, а також спрощення інтеграції існуючих монолітних систем в нову архітектуру

Згідно SOA сервіси реєструються на спеціальних сервісах і будь-яка команда розробників, якій потрібен доступ може знайти їх та використовувати.

Часто реалізація SOA покладається на використання централізованого програмного компонента для обміну даними – шину даних (Enterprise Service Bus).

Хід роботи

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати функціонал для роботи в розподіленому оточенні відповідно до обраної теми.
- Реалізувати взаємодію розподілених частин:
 - Для SOA додатків: реалізація сервісу, що надає послуги клієнтським застосуванням; викладання сервісу в хмару або підняття у вигляді Web Service на локальній машині; використання токенів для передачі даних про автентифікації, двостороннє шифрування.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє спроектовану архітектуру. Навести фрагменти програмного коду, які є суттєвими для відображення реалізованої архітектури.

Діаграма класів патерну SOA

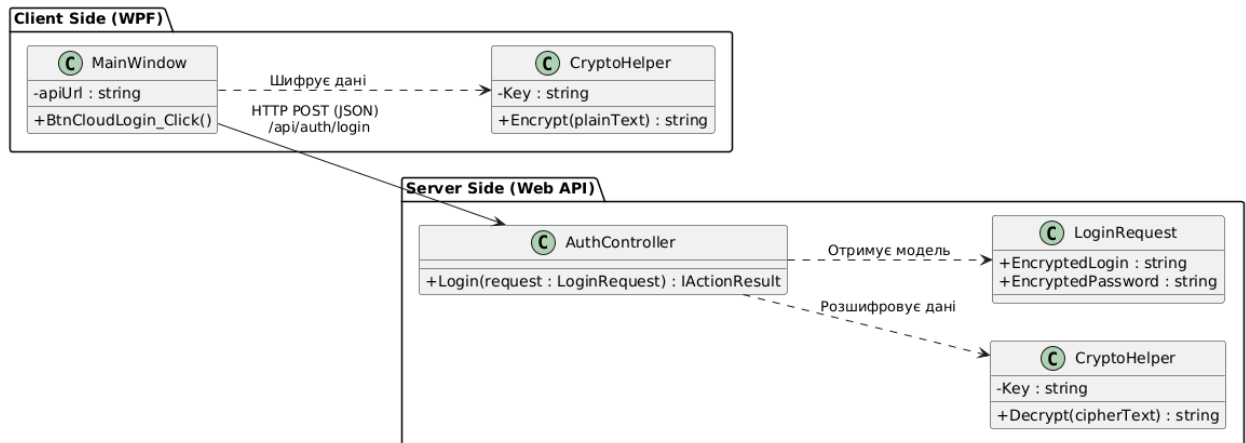


Рис. 1. Діаграма класів патерну SOA

Рівень клієнта

MainWindow – головний клас інтерфейсу, формує HTTP-запит

CryptoHelper – статичний допоміжний клас, відповідає за шифрування логіна та пароля перед відправкою до API

Рівень API

AuthController – приймає дані та повертає JWT-токен

LoginRequest – вбудована структура що містить поля з зашифрованими логіном і паролем

CryptoHelper – статичний допоміжний клас, аналогічний тому що знаходиться на рівні клієнта, відповідає за дешифровку вхідних даних

Код програми

MindMapAppAPI/controllers/**CryptoHelper** (аналогічний клас створено в MindMapApp)

```
using System.Security.Cryptography;
```

```
using System.Text;
```

```
namespace MindMapApp.Server.Helpers
```

```
{
```

```
    public static class CryptoHelper
```

```
    {
```

```
        //хардкод ключа шифрування
```

```

        private static readonly string Key =
"E546C8DF278CD5931069B522E695D4F2";

        public static string Decrypt(string cipherText)
        {
            using (var aes = Aes.Create())
            {
                aes.Key = Encoding.UTF8.GetBytes(Key);
                aes.IV = new byte[16];
                var decryptor = aes.CreateDecryptor(aes.Key, aes.IV);

                using (var ms = new
MemoryStream(Convert.FromBase64String(cipherText)))
                using (var cs = new CryptoStream(ms, decryptor,
CryptoStreamMode.Read))
                using (var sr = new StreamReader(cs))
                {
                    return sr.ReadToEnd();
                }
            }
        }
    }
}

```

MindMapAppAPI/controllers/**AuthController**

```

using Microsoft.AspNetCore.Mvc;
using MindMapApp.Server.Helpers;

```

```

namespace MindMapApp.Server.Controllers
{
    [ApiController]
    [Route("api/[controller]")]

```

```
public class AuthController : ControllerBase
{
    [HttpPost("login")]
    public IActionResult Login([FromBody] LoginRequest request)
    {
        try
        {
            string login = CryptoHelper.Decrypt(request.EncryptedLogin);
            string password = CryptoHelper.Decrypt(request.EncryptedPassword);
            // перевірка обл. даних, хардкод
            if (login == "admin" && password == "12345")
            {
                return Ok(new { Token = Guid.NewGuid().ToString(), Message =
"Вітаємо!" });
            }
            return Unauthorized("Невірний логін або пароль");
        }
        catch
        {
            return BadRequest("Помилка розшифрування");
        }
    }
}

public class LoginRequest
{
    public string EncryptedLogin { get; set; }
    public string EncryptedPassword { get; set; }
}
}
```

MindMapAppAPI/controllers/**MapsController**

using Microsoft.AspNetCore.Mvc;

using Microsoft.EntityFrameworkCore;

using MindMapApp.Entities;

using MindMapAppAPI.Data;

namespace MindMapApp.Server.Controllers

{

[ApiController]

[Route("api/[controller]")]

public class MapsController : ControllerBase

{

private readonly ApplicationDbContext _context;

public MapsController(ApplicationDbContext context)

{

_context = context;

}

[HttpGet]

public async Task<IActionResult> GetMaps()

{

var maps = await _context.MindMaps

.Select(m => new { m.Id, m.Title })

.ToListAsync();

return Ok(maps);

}

[HttpGet("{id}")]

public async Task<IActionResult> GetMapDetails(int id)

{

```

        var map = await _context.MindMaps
            .Include(m => m.Nodes)
            .Include(m => m.Regions).ThenInclude(r => r.Nodes)
            .FirstOrDefaultAsync(m => m.Id == id);

        if (map == null) return NotFound();
        return Ok(map);
    }

    [HttpPost]
    public async Task<IActionResult> CreateMap([FromBody] MindMap map)
    {
        _context.MindMaps.Add(map);
        await _context.SaveChangesAsync();
        return Ok(map);
    }
}

```

MindMapAppAPI/**Program**

```

using Microsoft.EntityFrameworkCore;
using MindMapAppAPI.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

```



```
var connectionString =  
"Server=(localdb)\\mssqllocaldb;Database=MindMapDb;Trusted_Connection=True;  
MultipleActiveResultSets=true";
```

```
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlServer(connectionString));
```

```
var app = builder.Build();  
if (app.Environment.IsDevelopment())  
{  
    app.UseSwagger();  
    app.UseSwaggerUI();  
}
```

```
app.UseHttpsRedirection();  
app.MapControllers();  
app.Run();
```

MindMapApp/**MainWindow** (додано метод)

```
private async void BtnCloudLogin_Click(object sender, RoutedEventArgs e)  
{  
    string apiUrl = "https://localhost:7225/api/auth/login";
```

```
    // хардкод логіна і пароля для наочності
```

```
    string rawLogin = "admin";  
    string rawPassword = "12345";
```

```
    try  
    {  
        var payload = new  
        {
```

```

        EncryptedLogin = CryptoHelper.Encrypt(rawLogin),
        EncryptedPassword = CryptoHelper.Encrypt(rawPassword)
    };

    string json = JsonSerializer.Serialize(payload);
    var content = new StringContent(json, Encoding.UTF8, "application/json");
    using (var client = new HttpClient())
    {
        var response = await client.PostAsync(apiUrl, content);
        if (response.IsSuccessStatusCode)
        {
            var responseString = await response.Content.ReadAsStringAsync();
            MessageBox.Show($"Успішна авторизація!\nВідповідь сервера:\n{responseString}", "SOA Client");
            BtnCloudLogin.Background =
                System.Windows.Media.Brushes.LightGreen;
        }
        else
        {
            MessageBox.Show($"Помилка: {response.StatusCode}", "SOA Client");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Не вдалося з'єднатися з сервером:\n{ex.Message}\n\n", "Помилка");
    }
}

```

Висновок: В ході виконання лабораторної роботи було реалізовано сервіс-орієнтовну архітектуру шляхом створення Web API сервісу на платформі ASP.NET Core. Сервіс виконує функцію провайдера ідентифікації та взаємодіє з клієнтським застосунком через протокол HTTP. Для забезпечення безпеки в розподіленому середовищі імплементовано механізм двостороннього симетричного шифрування

Питання до лабораторної роботи

1. Що таке клієнт-серверна архітектура?
Модель взаємодії в комп'ютерних мережах, де завдання розподіляються між сервером та клієнтом, причому клієнт завжди ініціює з'єднання та надсилає запити, а сервер очікує їх, обробляє та повертає відповіді.
2. Розкажіть про сервіс-орієнтовану архітектуру
Модульний підхід до розробки програмного забезпечення, що базується на використанні розподілених, слабо пов'язаних сервісів зі стандартизованими інтерфейсами, які можуть взаємодіяти між собою через мережу для виконання бізнес-завдань незалежно від платформ та мов програмування.
3. Якими принципами керується SOA?
Слабка зв'язаність, наявність стандартизованого контракту взаємодії, можливість багаторазового використання сервісів, абстрагування логіки та відсутність стану у комунікації.
4. Як між собою взаємодіють сервіси в SOA?
Через обмін повідомленнями за визначеними протоколами, використовуючи чітко описані інтерфейси, при цьому часто застосовується корпоративна сервісна шина як посередник для маршрутизації, перетворення форматів даних
5. У чому полягають переваги та недоліки клієнт-серверної моделі?
Переваги – централізація даних, що спрощує адміністрування та контроль безпеки, а також легкість масштабування серверної частини. Недоліки – наявність єдиної точки відмови, висока вартість серверного обладнання та залежність швидкодії від пропускну здатності мережі.
6. У чому полягають переваги та недоліки однорангової моделі взаємодії?
Переваги – висока відмовостійкість, легка масштабованість та відсутність потреби в дорогому центральному сервері. Недоліки – складність керування та адміністрування, проблеми з безпекою даних та відсутність гарантії доступності конкретного файлу чи ресурсу в будь-який момент часу.

7. Що таке мікро-сервісна архітектура?

Сучасна еволюція SOA, при якій застосунок будується як набір невеликих, автономних сервісів, кожен з яких виконує одну конкретну бізнес-функцію, має власну базу даних, розгортається незалежно від інших і спілкується через легковагові протоколи.

8. Які протоколи використовуються для обміну даними в мікросервісній архітектурі?

Синхронні HTTP/HTTPS (REST, GraphQL) та gRPC для швидкої взаємодії, а також асинхронні протоколи черг повідомлень (AMQP, MQTT) для забезпечення слабкої зв'язаності.

9. Чи можна назвати підхід сервіс-орієнтованою архітектурою, коли ми в проєкті між шаром веб-контролерів та шаром доступу до даних реалізуємо шар бізнес-логіки у вигляді сервісів?

Ні, оскільки сервіси знаходяться в одному адресному просторі, працюють в одному процесі та спілкуються через виклики методів у пам'яті, а не через мережу. SOA вимагає фізичного або логічного розподілення та незалежності компонентів.